

Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics

Technical Report: Formal Development and Proofs

Anonymous Author(s)

Abstract

This technical report is the formal companion to the paper *Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics* [1], which presents STEP-CHECK, a static verifier for the Amazon States Language (ASL). The paper states three guarantees and gives their intuition; here we give the complete development behind them: the abstract interpretation underlying the sound data-flow provenance analysis (domain, concretization, transfer functions, fixpoint, and the termination/convergence guard), the workflow type discipline for contracts and tpestate, and the downstream-aware Saga-compensation check, with full proofs of all three theorems. Theorems 1–3 below are numbered to match Theorems 1–3 of the main paper. We also give the SC4011 algorithm in full and note the conservative basis of the concurrency and temporal analyses. Section and code references (e.g. SC1101) follow the main paper; this report is released with the artifact and is self-contained with respect to the formal development, deferring only the empirical material (the execution oracle and the evaluation) to the main paper.

Contents

1	Scope and relation to the main paper	2
2	The data-flow abstract interpretation	2
2.1	The concrete property	2
2.2	Abstract domain	3
2.3	Concretization	3
2.4	Environments: document and variables	4
2.5	Transfer functions	4
2.6	The fixpoint	6
2.7	Reference checking	7
2.8	Nested machines	9
2.9	The dead-guard refinement (SC1110)	9
2.10	Coverage of the modeled fragment	9
2.11	External validity: independent repositories	10
3	The workflow type discipline	11
3.1	Signatures, judgment, and validity	11
4	Downstream-aware Saga compensation	12
4.1	Model of failure and the post-commit region	12
4.2	The SC4011 algorithm	13
4.3	Reverse-order chains	14
5	Conservative analyses: concurrency and temporal (notes)	14

1 Scope and relation to the main paper

The main paper keeps theorem statements and one-paragraph intuitions inline and defers their proofs and the supporting formalism here, so the body stays within venue limits. This report adds nothing to the claims; it discharges them.

We assume the ASL data plane as fixed in the paper’s background [8]. A workflow is a state machine over eight state types (**Task**, **Choice**, **Parallel**, **Map**, **Wait**, **Pass**, **Succeed**, **Fail**). A single JSON document is threaded between states and reshaped by the five-stage I/O pipeline $\text{InputPath} \rightarrow \text{Parameters/ItemSelector} \rightarrow \text{ResultSelector} \rightarrow \text{ResultPath} \rightarrow \text{OutputPath}$. A `.$` field reference reads a `JSONPath` [6] into the current document, and the runtime *fails* the execution if such a reference resolves to an absent field.

The development below is faithful to the implementation (module `passes/dataflow.rs` for Section 2, `passes/contract.rs` and `passes/typestate.rs` for Section 3, and `passes/compensation.rs` for Section 4); the execution oracle (`concrete.rs`) that empirically witnesses Theorem 1 is described in the main paper. Nested machines (**Map** iterators and **Parallel** branches) are treated in Remark 2.

Table 1 records, for each fact a check needs, whether ASL exposes it natively and where **STEPCHECK** obtains it otherwise (summarized in prose in the main paper’s background section).

Fact a check needs	In Asl?	Source in StepCheck
Control flow and terminals	yes	native structural pass
JSONPath data movement	yes	native data-flow pass
Contracts and typestate	no	sidecar / DSL / schema metadata
Idempotency, persistence, compensator	no	declared premises; optional warning inference
Shared resources and time bounds	partial	conservative native passes

Table 1: Facts needed by **STEPCHECK** and where they come from.

2 The data-flow abstract interpretation

The provenance analysis is a forward abstract interpretation [2] over a lattice of JSON *document and variable shapes*: it over-approximates, at each state, the document and workflow-variable store with which the state may be entered, and flags a reference that is absent on every such environment.

2.1 The concrete property

Fix a workflow with state set S . A concrete execution threads a JSON document *and* a store of workflow variables (written by **Assign**); write $\mathcal{D}$ for the set of JSON values. For a state s , let $\llbracket s \rrbracket$ be the set of *environments*—a document paired with a variable store—with which s is entered over all executions that reach s . A path $p = k_1.k_2.\dots.k_m$ is *present* in a JSON value d if d is an object with key k_1 whose value has k_2 , and so on to k_m ; otherwise p is *absent*. Each read site selects one source—the document, or, for a $\$name$ reference, the variable store. The analysis answers a single question soundly: *is the read path p absent in that selected source on every*

environment in $\llbracket s \rrbracket$? If so, the reference evaluated at s fails on every execution (a JSONPath path-not-found, an unresolved variable, or a JSONata `States.QueryEvaluationError`), and STEP CHECK reports SC1101.

2.2 Abstract domain

A *document shape* abstracts a set of documents by the field paths that *may* be present.

Definition 1 (Shapes). The set of shapes Σ is generated by

$$\sigma ::= \top \mid \text{Obj } r, \quad r : \text{Key} \rightarrow \Sigma \text{ a finite partial map.}$$

$\top$ is opaque (any field may be present). $\text{Obj } r$ constrains an object to the keys in $\text{dom}(r)$: every top-level key that *may* occur is listed, so a key outside $\text{dom}(r)$ is present on no represented *object*. The abstraction functions send a scalar to $\text{Obj } \emptyset$ (no fields) and an array to $\top$ (its elements are unconstrained); this is a choice of the abstraction, not a claim that $\gamma(\text{Obj } r)$ contains only objects—indeed $\gamma(\text{Obj } r)$ admits every non-object (Definition 4), so $\text{Obj } r$ is a constraint on the object case alone.

Definition 2 (Order and join). The join $\sqcup$ is

$$\top \sqcup \sigma = \sigma \sqcup \top = \top, \quad \text{Obj } r_1 \sqcup \text{Obj } r_2 = \text{Obj } r, \quad \text{dom}(r) = \text{dom}(r_1) \cup \text{dom}(r_2),$$

where $r(k) = r_1(k) \sqcup r_2(k)$ if $k \in \text{dom}(r_1) \cap \text{dom}(r_2)$, and $r(k) = r_i(k)$ if k occurs only in r_i . The induced order is $\sigma_1 \sqsubseteq \sigma_2 \iff \sigma_1 \sqcup \sigma_2 = \sigma_2$; equivalently $\sigma \sqsubseteq \top$ for all σ , and $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ iff $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$ for all $k \in \text{dom}(r_1)$.

Intuitively a shape is *higher* when it admits *more* documents: $\top$ is the top, $\text{Obj } \emptyset$ the most precise object shape, and the join is the least upper bound. The join is exactly the key-union of the implementation (clone the left record, then for each key of the right record join or insert).

Definition 3 (Definite absence). A path $p = k_1 \dots k_m$ is *definitely absent* in σ if descending σ by $k_1, \dots$ reaches some $\text{Obj } r$ with $k_i \notin \text{dom}(r)$ *before* encountering $\top$. Descent stops at $\top$: a path under $\top$ is never definitely absent.

2.3 Concretization

Definition 4 (Concretization γ). $\gamma : \Sigma \rightarrow \mathcal{P}(\mathcal{D})$ is

$$\gamma(\top) = \mathcal{D}, \quad \gamma(\text{Obj } r) = \{ d \in \mathcal{D} \mid d \text{ object} \Rightarrow \forall k \in \text{keys}(d). k \in \text{dom}(r) \wedge d(k) \in \gamma(r(k)) \}.$$

A non-object d (scalar, array) has no top-level keys, so it lies in $\gamma(\text{Obj } r)$ for every r .

Lemma 1 (Absence is captured). *If p is definitely absent in σ , then p is absent in every $d \in \gamma(\sigma)$.*

Proof. Induction on $|p|$. If $p = k_1 \dots$ and $\sigma = \text{Obj } r$ with $k_1 \notin \text{dom}(r)$, then by definition of γ no $d \in \gamma(\text{Obj } r)$ has top-level key k_1 , so p is absent in d . Otherwise $k_1 \in \text{dom}(r)$ and $k_2 \dots k_m$ is definitely absent in $r(k_1)$; for any $d \in \gamma(\text{Obj } r)$, either d lacks k_1 (so p absent) or $d(k_1) \in \gamma(r(k_1))$, and by the induction hypothesis $k_2 \dots k_m$ is absent in $d(k_1)$, so p is absent in d . Definite absence cannot pass through $\top$, so the case $\sigma = \top$ never arises along the descent. $\square$

Lemma 2 (γ is monotone, $\sqcup$ is sound). $\sigma_1 \sqsubseteq \sigma_2 \Rightarrow \gamma(\sigma_1) \subseteq \gamma(\sigma_2)$, and $\gamma(\sigma_1) \cup \gamma(\sigma_2) \subseteq \gamma(\sigma_1 \sqcup \sigma_2)$.

Proof. Monotonicity is by induction on shape structure: if $\sigma_2 = \top$ it is immediate; for $\text{Obj } r_1 \sqsubseteq \text{Obj } r_2$ we have $\text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $r_1(k) \sqsubseteq r_2(k)$, so any d admitted by $\text{Obj } r_1$ has $\text{keys}(d) \subseteq \text{dom}(r_1) \subseteq \text{dom}(r_2)$ and $d(k) \in \gamma(r_1(k)) \subseteq \gamma(r_2(k))$ by the induction hypothesis. Soundness of $\sqcup$ follows since $\sigma_i \sqsubseteq \sigma_1 \sqcup \sigma_2$ and γ is monotone. $\square$

2.4 Environments: document and variables

Current ASL threads not one but *two* pieces of state: the JSON document and a store of workflow *variables*, written by **Assign** and read as $\$name$ (JSONPath mode) or as $\$name$ inside a $\{\% \%\}$ expression (JSONata mode). Because a field can now travel through a variable instead of the document, provenance must track the variable store to stay sound: a read the analysis cannot see satisfied would otherwise be flagged as a missing document field. We abstract the store by a *single* shape—the record whose keys are the variable names that may be bound, each mapped to its own shape—drawn from the *same* lattice Σ . An abstract *environment* is then a pair

$$\mathcal{E} = \Sigma \times \Sigma, \quad e = (\sigma_d, \sigma_v),$$

with document component σ_d and variable-store component σ_v . Order, join, and concretization are componentwise: $(\sigma_d, \sigma_v) \sqsubseteq (\sigma'_d, \sigma'_v)$ iff $\sigma_d \sqsubseteq \sigma'_d$ and $\sigma_v \sqsubseteq \sigma'_v$; $(\sigma_d, \sigma_v) \sqcup (\sigma'_d, \sigma'_v) = (\sigma_d \sqcup \sigma'_d, \sigma_v \sqcup \sigma'_v)$; and

$$\gamma_{\mathcal{E}}(\sigma_d, \sigma_v) = \{ (d, \nu) \mid d \in \gamma(\sigma_d), \nu \in \gamma(\sigma_v) \},$$

a concrete environment (d, ν) pairing the document d with the variable store ν (itself a JSON object mapping names to values). The empty store $\text{Obj } \emptyset$ (no variable bound) seeds σ_v . Because $\mathcal{E}$ is a finite product of the finite-height lattice Σ , it is again finite-height, and Lemmas 1 and 2 lift componentwise. All transfers below act on environments; one that touches only the document leaves σ_v unchanged, and **Assign** touches only σ_v .

2.5 Transfer functions

Each stage is a monotone function on shapes. We write $\text{narrow}(\sigma, \pi)$ for re-rooting along a path expression π (used by **InputPath/OutputPath**), $\text{ctor}(v)$ for the record built from a constructor value v (used by **Parameters/ItemSelector/ResultSelector**), and $\text{place}(\sigma, \rho, \sigma_r)$ for merging a result shape σ_r at a **ResultPath** ρ .

Re-rooting. Write ε for the empty (absent) path and $k :: \text{rest}$ for the split of a path into its first key and remainder. Then $\text{narrow}(\sigma, \varepsilon) = \sigma$ (no path) and $\text{narrow}(\sigma, \$) = \sigma$ (the bare root); $\text{narrow}(\sigma, \text{null}) = \text{Obj } \emptyset$, sound because ASL maps an **InputPath/OutputPath** of **null** to the empty object $\{\} \in \gamma(\text{Obj } \emptyset)$. For a precise member path $\pi = k_1 \dots k_n$ —written either in dotted form or quoted-bracket member form— narrow descends via nav : $\text{nav}(\top, _) = \top$ and $\text{nav}(\text{Obj } r, k :: \text{rest}) = \text{nav}(r(k), \text{rest})$ if $k \in \text{dom}(r)$, and $= \top$ otherwise (a missing intermediate key yields $\top$, over-approximating). Any path outside this member-path fragment (wildcards, filters, functions, array indexes, the $\$\$$ context) yields $\top$; a non-string, non-**null** path value is treated as the identity ($\text{narrow}(\sigma, \cdot) = \sigma$), a case ASL deploy-time validation rejects and which is therefore unreachable in practice.

Constructors and literals. Two builders recurse over a JSON value v . The *constructor* $\text{ctor}(v)$ models a payload template (**Parameters/ItemSelector/ResultSelector**): for an object it builds $\text{Obj } r$ where a key “ $k.\$$ ” contributes $r(k) = \top$ (it *reads* an opaque value into field k), a static key k contributes $r(k) = \text{ctor}(v(k))$, a scalar leaf is $\text{Obj } \emptyset$, and an array is $\top$. The *literal* builder $\text{lit}(v)$ models verbatim data (a **Pass Result**), which ASL copies uninterpreted: it agrees with ctor *except* that it does no $.\$$ stripping—a key literally named “ $k.\$$ ” stays a key of that exact name. Distinguishing the two is necessary for soundness: for $\text{Result} = \{\text{“a.\$”}: v\}$ the concrete document has the top-level key “**a.\\$**”, which is *not* in $\text{dom}(\text{ctor}) = \{a\}$, so ctor would under-approximate the key set.

Result and merge. The result shape σ_{res} of a state, built from its effective input eff , is: $\top$ for an opaque `Map/Parallel` result and for a `Task` result with no modeled result shape; $ctor(\text{ResultSelector})$ when a `ResultSelector` pins it; a closed record when the optional result-shape model supplies a declared task-output contract or a fixed AWS service-result envelope; and for a `Pass`, $lit(\text{Result})$ if it carries a literal `Result`, else $ctor(\text{Parameters})$ if it carries only `Parameters`, else eff itself (a bare `Pass` forwards its effective input unchanged). Merging is

$$place(\sigma, \rho, \sigma_r) = \begin{cases} \sigma_r & \rho = \text{Default or } \rho = \$ \quad (\text{result replaces the document}), \\ \sigma & \rho = \text{Discard (null)} \quad (\text{result dropped, input kept}), \\ place_{in}(\sigma, \pi, \sigma_r) & \rho = \text{Path}(\pi), \end{cases}$$

where $place_{in}$ descends π , cloning each record and *preserving siblings*, and writes σ_r at the leaf; a $\top$ parent stays $\top$, and an unparseable π yields $\top$. (This is why a field can survive a task that never mentions it: a sub-path merge leaves the rest of the document intact.)

Variable assignment. An `Assign` block updates the variable store, *overwriting* any earlier binding of the same name (bindings from distinct control-flow paths are joined only where they merge, at the fixpoint). Write $bind(\sigma_v, k, \sigma)$ for the store that agrees with σ_v except that key k maps to σ (identity when $\sigma_v = \top$). In JSONPath mode an entry “ $k.\$$ ”: π binds $k \mapsto narrow(\sigma_{src}, \pi)$ and a static entry $k : v$ binds $k \mapsto ctor(v)$, where the source shape σ_{src} is the value ASL reads an `Assign` against for that state kind—the raw result ($\top$, unless result-shape modeling supplies a task result record) for `Task`, $\top$ for `Map/Parallel`, the `Pass` result shape, or the effective input eff for `Choice/Wait`. In JSONata mode an entry $k : v$ binds $k \mapsto jval(v)$ (below). The whole block $asgn(\sigma_v, \text{Assign})$ is the composition of these single-key binds.

JSONata expressions. A JSONata value—a string “ $\{ \% e \% \}$ ”, or a JSON object/array containing such strings—denotes a shape $jval$ against the current sources: the entry document, the state’s raw result, an opaque error object, and the variable store, each itself a shape. On the *modeled fragment* a *direct reference* $\$states.input.\pi$, $\$states.result.\pi$, $\$states.errorOutput.\pi$, or $\$name.\pi$ navigates (*nav*) the corresponding source shape at π ; an *object literal* $\{ 'k_1': e_1, \dots \}$ builds $Obj\{k_i \mapsto jval(e_i)\}$; a surrounding JSON object/array recurses into its members. Every other expression—functions such as $\$exists$, filters, arithmetic, string operators, and $\$states.context$ —yields $\top$. A modeled read thus resolves against exactly one source shape and an unmodeled one is opaque: the JSONata analogue of the simple-path fragment. This is what lets a field move through a variable in JSONata mode ($\text{Assign}\{k : \{ \% \$states.input.\pi \% \}\}$ then a later $\$k$ read) without a spurious finding, while still flagging a read of a name no path ever assigns.

State transfer and error edges. For a state with entry environment $e = (\sigma_d, \sigma_v)$, the output environment $out(e) = (\sigma'_d, \sigma'_v)$ updates both components. The variable store becomes $\sigma'_v = asgn(\sigma_v, \text{Assign})$ (identity when the state has no `Assign`). The document becomes

$$\sigma'_d = \begin{cases} jout(e) & \text{JSONata mode (query state),} \\ narrow(place(\sigma_d, \rho, \sigma_{res}), \text{OutputPath}) & \text{Task/Pass/Map/Parallel,} \\ narrow(eff, \text{OutputPath}) & \text{otherwise,} \end{cases}$$

with $eff = narrow(\sigma_d, \text{InputPath})$ the `InputPath`-selected input from which the result shape σ_{res} is built, and ρ the state’s `ResultPath`. The JSONPath result is merged into the *raw* entry document σ_d , *not* eff —exactly as ASL writes `ResultPath` into the pre-`InputPath` document, so a field the `InputPath` selection dropped still survives. In JSONata mode the successor document is the shape of the state’s `Output`, $jout(e) = jval(\text{Output})$ —an object literal closes the successor

shape precisely, and a query **Task** with no **Output** yields its modeled raw-result shape when result-shape modeling is enabled, otherwise $\top$ (as do **Map/Parallel**); ASL’s JSONata mode has no **InputPath/OutputPath** to re-root. A **Catch** edge from entry e propagates $catch(e, \rho) = (place(\sigma_d, \rho, \top), asgn(\sigma_v, Catch.Assign))$: the error object is opaque ($\top$), merged at the catch’s **ResultPath**, and the catcher’s own **Assign** (if any) updates the store. A **Retry** re-executes the same state on the same entry environment, so it leaves entry shapes unchanged and the analysis ignores it.

Lemma 3 (Local soundness). *Writing a subscript c for the concrete counterpart of each stage, every transfer is sound on its own arguments:*

(re-root) $d \in \gamma(\sigma) \Rightarrow narrow_c(d, \pi) \in \gamma(narrow(\sigma, \pi))$;

(build) *the concrete payload built over a document d from a template v lies in $\gamma(ctor(v))$, and from a literal v in $\gamma(lit(v))$;*

(merge) $d \in \gamma(\sigma) \wedge d_r \in \gamma(\sigma_r) \Rightarrow place_c(d, \rho, d_r) \in \gamma(place(\sigma, \rho, \sigma_r))$;

(jsonata) *if each source value lies in the γ of its source shape, the concrete value of a JSONata expression e lies in $\gamma(jval(e))$;*

(assign) $\nu \in \gamma(\sigma_v) \Rightarrow asgn_c(\nu, Assign) \in \gamma(asgn(\sigma_v, Assign))$;

(state/catch) $(d, \nu) \in \gamma_{\mathcal{E}}(e) \Rightarrow out_c(d, \nu) \in \gamma_{\mathcal{E}}(out(e))$ and $catch_c(d, \nu) \in \gamma_{\mathcal{E}}(catch(e, \rho))$.

Proof. By cases. *Re-rooting:* if $\sigma = \top$ or π is non-simple, $narrow(\sigma, \pi) = \top$ and $\gamma(\top) = \mathcal{D}$; for a simple path, nav keeps $d(k) \in \gamma(r(k))$ by Definition 4, and a missing intermediate key yields $\top$ (vacuously sound); $narrow(\sigma, \text{null}) = \text{Obj } \emptyset$ is sound because the selected concrete value is the empty object $\{\}$, which has no keys and hence lies in $\gamma(\text{Obj } \emptyset)$. *Constructor:* the concrete payload object has top-level keys exactly the template’s keys $\subseteq \text{dom}(ctor(v))$; a $.\$$ key binds a runtime-read value in $\gamma(\top)$, and a static key’s value lies in $\gamma(ctor(v(k)))$ by induction. *Literal:* the concrete value is v copied verbatim, whose keys are exactly $\text{dom}(lit(v))$ —including any literal $.\$$ key—and each child lies in the corresponding γ by induction. *Result:* an opaque result lies in $\gamma(\top)$; a pinned **ResultSelector** reduces to the constructor case; a bare **Pass** forwards eff , and $d \in \gamma(eff)$ by hypothesis. *Merge:* $\rho \in \{\text{Default}, \$\}$ replaces, and the concrete output is the result, which lies in $\gamma(\sigma_r)$; $\rho = \text{Discard}$ keeps the input; for $\rho = \text{Path}(\pi)$, $place_{in}$ writes σ_r at π and preserves siblings, matching the concrete merge, and a $\top$ parent (or unparseable π) stays $\top$ (sound). *JSONata:* a modeled direct reference navigates one source shape and is sound by the re-root case (nav is $narrow$ ’s descent); an object literal is the constructor case over $jval$ -typed children; every unmodeled expression yields $\top$, whose γ admits the concrete value. *Assign:* each *bind* replaces one store key by a shape that—by the (build), (jsonata), or (re-root) case—over-approximates the concrete bound value while leaving the other keys’ admission unchanged, so the updated store lies in γ of the updated store shape; composing the entries yields $asgn$. *State transfer* pairs the document update (the JSONPath composition above, or $jout$ in JSONata mode) with the variable update $asgn$; since $\gamma_{\mathcal{E}}$ is the componentwise conjunction, soundness of each component gives soundness of the pair. *Catch* places $\top$ (the error object) at ρ in the document—sound since $\gamma(\top) = \mathcal{D}$ —and applies $asgn$ for the catcher’s **Assign**. $\square$

2.6 The fixpoint

The analysis maintains a *partial* map $\Phi : S \rightharpoonup \mathcal{E}$, seeding only the start state with $e_0 = (\sigma_0, \text{Obj } \emptyset)$ (document seed as below, empty variable store); a state acquires an environment the first time an edge reaches it, and a state never reached stays outside $\text{dom}(\Phi)$ and is read as $(\top, \top)$ at check time (an unreachable state constrains nothing, and reporting against $\top$ never fires). One

round updates, for every $s \in \text{dom}(\Phi)$ with current entry $\Phi(s)$: for each normal successor t , $\Phi(t) \leftarrow \Phi(t) \sqcup \text{out}(\Phi(s))$ (inserting $\text{out}(\Phi(s))$ when $t \notin \text{dom}(\Phi)$); for each **Catch** target u with result path ρ , $\Phi(u) \leftarrow \Phi(u) \sqcup \text{catch}(\Phi(s), \rho)$. Updates are in place (Gauss–Seidel) in an unspecified state order; the least fixpoint, once reached, is independent of that order. Rounds repeat until a round changes nothing.

Lemma 4 (Widened convergence). *The forward iteration with depth- k widening is monotone and reaches a post-fixpoint on every workflow topology. The implementation uses $k = 12$ and records the safety bound $N = |S| \cdot (\kappa_0 + 2) \cdot (k + 1) + 2$ rounds, where κ_0 is the implementation’s key count. If the safety bound were exceeded because of an implementation bug, all known entries would be lifted to $(\top, \top)$ before checking, preserving soundness by losing precision rather than reporting from an under-approximation.*

Proof. Every normal or catch-edge update is a join, so each $\Phi(s)$ is non-decreasing (Lemma 2). Let κ be the workflow’s finite *key universe*: keys occurring in constructors (**Parameters**, **Result**, **ResultSelector**, **ItemSelector**), in **Assign/Arguments/Output** templates or JSONata object literals, in declared input or output schemas, and as segments of **ResultPath** and catch **ResultPath** fields. Define W_k over shapes by replacing every record nested deeper than k by $\top$ and recursing otherwise. W_k is extensive and monotone: for every shape σ , $\gamma(\sigma) \subseteq \gamma(W_k(\sigma))$, and if $\sigma_1 \sqsubseteq \sigma_2$ then $W_k(\sigma_1) \sqsubseteq W_k(\sigma_2)$. The implementation applies W_k to every propagated environment and again after each join.

After widening, every environment component ranges over records of depth at most k over the finite key universe, plus $\top$; the product environment lattice therefore has finite height. A monotone in-place iteration over a finite-height lattice stabilises at a post-fixpoint independent of state order. Cyclic re-embedding can still push a growing tail through a loop, but once the tail passes depth k it becomes $\top$, so the chain stops growing. The emergency lift to $(\top, \top)$ is also an over-approximation: subsequent propagation can only join smaller transfer results into $\top$ entries, so no definite-absence report can arise from the lifted region. $\square$

Remark 1 (Empirical headroom). The widened bound is deliberately conservative. On the current corpus—193 AWS workflows, 66 CNCF workflows, and two DSL workflows, including nested machines and both native and typed seeding—all 790 data-flow fixpoints converge, with a maximum of 7 rounds and a worst observed bound utilization of $2/54$ (3.7%). The synthetic scale sweep remains near-linear; acyclic chains converge in one forward pass by construction.

2.7 Reference checking

At the widened fixpoint (Lemma 4), the analysis checks, at each state s with entry environment $\Phi(s) = (\sigma_d, \sigma_v)$ and effective document $\text{eff} = \text{narrows}(\sigma_d, \text{InputPath})$, the reads the runtime evaluates and aborts on when absent. These fall into three classes.

(i) *JSONPath payload reads.* The $.\$$ keys (at any depth) of **Parameters** and **ItemSelector** and a **Map**’s **ItemsPath**, resolved against eff ; and the $.\$$ keys of a JSONPath **Assign**, resolved against that state’s assign source. A reference $\$name.\pi$ that names a workflow *variable* (a leading $\$$ not followed by “.”, hence not a document path $\$. \dots$) is resolved against the store σ_v rather than eff —so a field that travelled through a variable is seen where it is read, and only a variable no path assigns is flagged.

(ii) *Modeled JSONata reads.* In a JSONata state, a field-value string whose *entire* body is a direct reference ($\$states.\text{input}.\pi$, $\$states.\text{result}.\pi$, $\$states.\text{errorOutput}.\pi$, or $\$name.\pi$)—in **Arguments**, **Output**, **Assign**, or a **Choice Condition**—resolved against its source shape (effective document, raw result, error object, or store). Such an expression returns **undefined** on a missing path, which ASL raises as **States.QueryEvaluationError** [8] (“failure to return a result”)—a definite runtime fault, exactly the SC1101 contract. A reference nested inside a JSONata *object constructor* $\{ 'k': e \}$ is deliberately *not* in this class: JSONata omits a key whose

value is undefined, so the constructor yields a value and does not fail; such a reference shapes the successor document (§2) but is never itself flagged.

(iii) *Excluded (opaque) reads.* Any string not beginning with \$, the context object `$$/$states.context`, `States.*` intrinsics, the bare root \$, any JSONPath outside the simple-path fragment, and any JSONata expression outside the modeled direct-reference/object fragment (functions such as `$exists`, filters, arithmetic). All resolve to $\top$ and never fire.

For a checked reference to a path p against its selected source shape σ (document or variable store), `STEPCHECK` reports `SC1101` iff p is definitely absent in σ (the lookup returns `Missing`; a path into a $\top$ or otherwise unconstrained region returns `Maybe` and is never flagged). A JSONPath `Choice` operand is handled as a *warning*, not an error; see Section 2.9.

Assumption 1 (Runtime resolution on non-objects). The ASL JSONPath engine resolves a dotted reference $k_1 \dots k_m$ by successive object-key lookup: applying a key to a non-object (scalar or array) fails with a path-not-found error, and a numeric segment denotes a property name rather than an array index [8, 6]. Under this assumption the concrete *present* relation of Section 2 (object-key descent) coincides with runtime resolution, and γ 's admission of arbitrary non-objects under `Obj r` is sound: a reference descending into a non-object simply fails, exactly as “absent” predicts.

Theorem 1 (No false positives). *If `STEPCHECK` reports `SC1101` for a path p at a state s , then in every concrete execution that reaches s , p is absent from the document or variable store that the read site selects, so the ASL runtime fails the reference—a JSONPath path-not-found, an unresolved variable, or a JSONata `States.QueryEvaluationError`. Equivalently, the analysis never flags a reference some execution could satisfy.*

Proof. First, the fixpoint over-approximates the reachable environments: for every s and every reachable $(d, \nu) \in \llbracket s \rrbracket$ (document d paired with variable store ν), $(d, \nu) \in \gamma_{\mathcal{E}}(\Phi(s))$. Induction on the length of the execution prefix reaching s . *Base:* the seed $e_0 = (\sigma_0, \text{Obj } \emptyset)$ satisfies it—the document part as before (native tier $\sigma_0 = \top$ and $\gamma(\top) = \mathcal{D}$; typed tier $\sigma_0 = \text{Obj } r_0$ over the declared input fields, under the closed-world reading of the contract), and the initial store is empty, so $\nu_0 = \{\} \in \gamma(\text{Obj } \emptyset)$. Closed task-result records are sound relative to the declared output contract or fixed AWS service envelope they model. *Step:* suppose s is reached from s' . On a normal transition the concrete environment is $\text{out}_c(d', \nu')$ for some $(d', \nu') \in \llbracket s' \rrbracket$; by the induction hypothesis $(d', \nu') \in \gamma_{\mathcal{E}}(\Phi(s'))$, by Lemma 3 (state/catch, which composes the document transfer with the `Assign` store update) $\text{out}_c(d', \nu') \in \gamma_{\mathcal{E}}(\text{out}(\Phi(s')))$, and since the round joined $\text{out}(\Phi(s'))$ into $\Phi(s)$, by the componentwise lift of Lemma 2 the concrete environment lies in $\gamma_{\mathcal{E}}(\Phi(s))$. A `Catch` transition is identical using *catch*, which places $\top$ in the document and applies the catcher's `Assign`. The `InputPath` narrowing at s is sound (Lemma 3), so effective documents lie in $\gamma(\text{eff})$ and stores in $\gamma(\sigma_v)$.

Now suppose `SC1101` fires for p at s , against its selected source shape σ —the effective document eff for a document read, or the store σ_v for a variable read. The check fires only when p is definitely absent in σ , so by Lemma 1 no value in $\gamma(\sigma)$ contains p ; since every concrete source at s lies in $\gamma(\sigma)$, p is absent on all of them. A JSONPath payload read then fails by Assumption 1 (path-not-found); a JSONPath variable read has no binding to resolve; and a modeled JSONata direct reference evaluates to `undefined`, which ASL raises as `States.QueryEvaluationError`. In every case the runtime aborts the reference. A finding is emitted only when the machine converged (Lemma 4), i.e. at a genuine fixpoint, so the conclusion holds whenever `SC1101` is reported. $\square$

Corollary 1 (Native unconditionality). *On the native tier ($\sigma_0 = \top$), Theorem 1 holds for an arbitrary execution input. On the typed tier it holds relative to the closed-world reading of the declared input schema; when result-shape modeling is enabled it also holds relative to the closed-world reading of declared task-output schemas and fixed AWS service-result envelopes.*

Where inputs or results are open, the native opaque shape is the sound fallback. (The typed seed also admits a second closed-world source: when a workflow declares no input schema but its start task declares an input contract, the start seed is that contract’s field record.)

2.8 Nested machines

A **Map**’s iterator and a **Parallel**’s branches are themselves state machines, each analysed by the same procedure under a seed derived from the enclosing state’s entry.

Remark 2 (Nested-machine seeds). The nested analysis runs under seeds derived from the enclosing machine’s widened fixpoint; if the safety guard ever lifted an enclosing entry to $\top$, the nested seed is correspondingly opaque and conservative. Each **Map** iteration is seeded with *ctor*(**ItemSelector**) when an **ItemSelector** is declared and $\top$ otherwise; each **Parallel** branch is seeded with *ctor*(**Parameters**) when the state declares **Parameters** and with the effective input *eff* = *narrow*($\Phi(s)$, **InputPath**) otherwise—exactly the document ASL hands each branch. Each seed over-approximates the corresponding per-iteration/per-branch document (Lemma 3), so Theorem 1 extends unconditionally to findings emitted inside a nested machine.

2.9 The dead-guard refinement (SC1110)

The same shapes support a path-sensitive *warning*. Unlike a **.\$** payload reference, a **Choice** comparator is a *guard* evaluated in rule order: the runtime tries each rule until one matches and follows its branch. A value comparator on an unresolvable operand does not silently return false—ASL raises a **States.Runtime** error [8], which is precisely why **IsPresent** exists. We nonetheless report only a *warning*, not an SC1101 error, for two reasons: the rule may never be reached (an earlier rule can match first and divert control), so we cannot promise a runtime abort; and the finding’s real content is that a branch whose guard *requires* a definitely absent field can never be taken—dead logic. SC1110 fires when a **Choice** rule’s condition requires presence of a field definitely absent in the state’s effective (post-**InputPath**) shape. Presence is required by value comparators, **IsNull**, the type predicates, and **IsPresent:true**; it is *not* required by **IsPresent:false**. The check is conservative on Boolean structure: a conjunction is dead if any conjunct is dead, a disjunction only if *all* disjuncts are dead, and a negation is never flagged. Soundness of the underlying “definitely absent” judgment is exactly Lemma 1.

2.10 Coverage of the modeled fragment

The analysis resolves precisely exactly the reads inside the member-path fragment (**.\$**, **\$.a**, **\$.a.b**, **\$.a’**); every operand outside it lifts to $\top$ and can never produce a finding. To quantify how much of real ASL this covers, the **stepcheck path-coverage** command classifies every field-read a workflow evaluates—the **.\$** operands of **Parameters/ItemSelector/Assign** and a **Map**’s **ItemsPath**, i.e. the read sites the SC1101 check consumes—into the dotted/quoted-member fragment, the whole-document **.\$**, complex JSONPath (wildcard/filter/function/array-index), the **\$\$** context object, **\$.name** workflow-variable reads, and **States.*** intrinsics. Table 2 reports the split over the AWS corpus, the CNCF examples, and the independent repositories of Section 2.11. JSONata **{% %}** reads form a separate modeled fragment—direct references and object literals (§2)—and are not counted in this JSONPath surface.

Of the genuine document-field references, 92–95% fall inside the modeled member fragment; only 6.6% (combined) use wildcard, filter, array-index, or other nonmember syntax that the analysis soundly abstracts to $\top$. No workflow in these JSONPath corpora issues a **\$.name** variable read (the **\$.var** column is zero); the variable and JSONata-fragment provenance is exercised instead by the regression suite and the JSONata-mode workflows in the corpus. The CNCF corpus exercises almost no **.\$** reads because Serverless Workflow uses runtime expressions rather

Table 2: Field-read coverage by the modeled JSONPath fragment (`stepcheck path-coverage`). “Doc. reads” are the genuine document-field references (member + whole-\$ + complex), excluding the \$\$ context and `States.*` operands, which are not document references and cannot miss a field. “Precise” is dotted or quoted-member access + whole-\$.

Corpus	Reads	Dotted	Whole-\$	Complex	\$\$	\$var	States.*	Precise / doc. reads
AWS (193)	1168	596	146	61	225	0	140	742/803 = 92.4%
CNCF (66)	2	0	0	0	0	0	2	— (runtime expr.)
Independent (95)	420	296	57	17	20	0	30	353/370 = 95.4%
Combined	1590	892	203	78	245	0	172	1095/1173 = 93.4%

than JSONPath provenance, consistent with the ASL-specific data-flow checks staying silent there.

2.11 External validity: independent repositories

Beyond the three aws-samples collections that make up the main corpus, we ran STEP CHECK unmodified on **95** ASL definitions mined (`eval/mine_wild.js`) from **16** public repositories outside them—50 domain workflows across 13 repositories, the remainder validator/library fixtures. Two findings from the sound and conservative native tiers illustrate that the analyses are not overfit to AWS’s samples.

A sound SC1101 in the wild. In a credit-card *transaction-reversal* workflow, a Map state `ApplyReversalEntries` iterates `ItemsPath: $.ledgerEntries`. The start state `SetReversalDefaults` is a Pass whose `Parameters` reconstruct the document as a closed record of seven fields (`transactionId`, `amount`, `currency`, `accountId`, `reason`, `reversalId`, `reversalStatus`); `ledgerEntries` is not among them, and the two intervening states merge only their own results under `$.validation` and `$.screening`. By the transfer functions of Section 2, the shape at `ApplyReversalEntries` is a closed record with `ledgerEntries` $\notin$ dom, so `present($.ledgerEntries)` is definitely false and SC1101 fires soundly—at run time the Map fails to resolve `ItemsPath`. Because `SetReversalDefaults` *reconstructs* the document rather than augmenting it, even a caller that supplies `ledgerEntries` has it dropped; an adjacent-task field-subset check, which never models the `Parameters` reconstruction, cannot see this.

A conservative SC5001 in the wild. In an IoT firmware-upgrade workflow, a `Parallel` state runs three branches that each write the DynamoDB table `FWUTaskTokens` (`PutItem`); the branches receive document copies, so the shared external table is the only race, and the concurrency pass reports the provable last-writer-wins collision.

Declared-tier evidence. The declared tier is silent on the wild corpus only because no workflow ships a compensator. Supplying the missing facts recovers it: for an independent payment-settlement pipeline, a sidecar declaring the payment authorization and the settlement as persistent effects (both are durable money movements) promotes STEP CHECK’s finding from an inferred warning to a declared-tier *error*—SC4001 on the uncompensated `FlagFraudulentPayment` and SC4010 on `AuthorizePayment/SettlePayment`, whose `Catch` paths only reach a Fail state, so a settlement failure leaves an authorized hold that is never voided. The sidecar and reproduction command ship in the artifact (`corpus/wild-annot/`).

3 The workflow type discipline

3.1 Signatures, judgment, and validity

Let Γ collect the declared task signatures. Each task T has a *contract* $A \rightarrow B$ (it consumes the field-record A and produces B) and a *typestate* [4, 5] transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ over business states S_B ; we bundle the two into one signature $T : (A, s_{\text{in}}^T) \rightarrow (B, s_{\text{out}}^T)$. The well-typedness judgment $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ carries a *pair* type—a field-record and a business state—and is generated by:

$$\begin{array}{c} \frac{(T : (A, s_{\text{in}}) \rightarrow (B, s_{\text{out}})) \in \Gamma}{\Gamma \vdash T : (A, s_{\text{in}}) \Rightarrow (B, s_{\text{out}})} \text{ (TASK)} \quad \frac{\Gamma \vdash T : (A, s) \Rightarrow (B, s') \quad A \subseteq A'}{\Gamma \vdash T : (A', s) \Rightarrow (B, s')} \text{ (SUB)} \\[10pt] \frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (B, s') \Rightarrow (C, s'')}{\Gamma \vdash T_1; T_2 : (A, s) \Rightarrow (C, s'')} \text{ (SEQ)} \\[10pt] \frac{\Gamma \vdash T_1 : (A, s) \Rightarrow (B, s') \quad \Gamma \vdash T_2 : (A, s) \Rightarrow (B, s')}{\Gamma \vdash \text{Choice}(T_1, T_2) : (A, s) \Rightarrow (B, s')} \text{ (CHOICE)} \end{array}$$

The width-subsumption rule (SUB) widens the *available* input fields—a task needing A still runs when more fields $A' \supseteq A$ are present—and leaves the business state untouched. It is what lets the field component be matched by *containment* while the typestate component is matched by *equality*, mirroring the two checks below; without it the SEQ rule would force the middle field-record to match exactly, which the tool does not require.

The judgment is read over the workflow’s *task-adjacency* structure: STEPCheck projects the control-flow graph onto its **Task** states, following *normal* edges through the *transparent* states (**Choice**, **Pass**, **Wait**) that carry no contract, so $T_1; T_2$ denotes a task immediately followed—through any transparent states—by another and **Choice**(T_1, T_2) a branch whose arms rejoin. **Catch** (error) edges are *not* projected, and **Map/Parallel** states are opaque adjacency endpoints whose sub-machines are checked separately, so the discipline governs the normal-flow task graph only. A declared *protocol* $\delta \subseteq S_B \times S_B$ lists the legal business-state transitions, and $\text{Valid}(W) \iff \forall T \in W. (s_{\text{in}}^T, s_{\text{out}}^T) \in \delta$.

The checks are edge-local. For each task-adjacency $T_1 \rightarrow T_2$ whose endpoints both carry the relevant declarations: the typed-contract check SC1010 fires iff some required field of T_2 is missing from T_1 ’s produced fields ($\text{in}(T_2) \not\subseteq \text{out}(T_1)$); the typestate check SC2001 fires iff $s_{\text{out}}^{T_1} \neq s_{\text{in}}^{T_2}$; and SC2002 fires iff $(s_{\text{in}}^{T_2}, s_{\text{out}}^{T_2}) \notin \delta$ when a non-empty protocol is declared. A **Choice** imposes the predecessor’s premise on each branch head. An edge missing an endpoint declaration is passed over—the checks are vacuous there—so the guarantee below is stated for *fully declared* workflows.

Theorem 2 (Declared-tier consistency). *Let W be fully declared: every **Task** carries a contract and both typestate labels, and a protocol δ is declared. Then SC1010, SC2001, and SC2002 emit no finding iff*

$$\begin{array}{l} \text{in}(T_2) \subseteq \text{out}(T_1) \text{ and } s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2} \text{ for every adjacency } T_1 \rightarrow T_2, \\ \text{and } (s_{\text{in}}^T, s_{\text{out}}^T) \in \delta \text{ for every task } T. \end{array} \quad (*)$$

The three checks are therefore a sound and complete decision procedure for (): every W satisfying (*) is accepted, and every violation is reported by a finding naming the offending edge (SC1010/SC2001) or task (SC2002). On the series-parallel fragment—acyclic task graphs built from sequencing and fully rejoining binary **Choice** whose arms share a common branch-point and rejoin pair type—condition (*) is moreover equivalent to derivability $\Gamma \vdash W : (A, s) \Rightarrow (B, s')$ together with $\text{Valid}(W)$, so on that fragment the checks decide well-typedness in the calculus above.*

Proof. The equivalence “no finding $\iff (*)$ ” is immediate: each check is exactly the negation of one clause of $(*)$, evaluated independently per edge (per task for SC2002), and full declaration leaves no edge vacuous. This holds for an arbitrary normal-flow task graph, cyclic ones included—a back-edge is simply another adjacency subject to $(*)$.

For the series-parallel fragment we show $(*) \iff \Gamma \vdash W : (A, s) \Rightarrow (B, s') \wedge \text{Valid}(W)$. ($\Leftarrow$) A derivation types each adjacency by SEQ, whose shared middle type (B, s') forces $\text{in}(T_2) \subseteq \text{out}(T_1)$ on the field component (a preceding SUB step only enlarges the available fields) and $s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2}$ on the typestate component; CHOICE imposes the same shared type on both arms, extending the equalities across a branch and its rejoin. $\text{Valid}(W)$ supplies the protocol clause, giving $(*)$. ($\Rightarrow$) Assume $(*)$. Build the derivation bottom-up: TASK gives each task $T : (\text{in}(T), s_{\text{in}}^T) \Rightarrow (\text{out}(T), s_{\text{out}}^T)$. For a sequential adjacency $T_1; T_2$ take the middle type $(B, s') = (\text{out}(T_1), s_{\text{out}}^{T_1})$: by $(*)$, $\text{in}(T_2) \subseteq \text{out}(T_1) = B$ and $s_{\text{in}}^{T_2} = s_{\text{out}}^{T_1} = s'$, so SUB lifts the TASK axiom of T_2 to $(B, s') \Rightarrow (\text{out}(T_2), s_{\text{out}}^{T_2})$ and SEQ applies. A fragment CHOICE types both arms at the common branch-point type (A, s) and common rejoin type (B, s') (each arm as above), so CHOICE applies. The result derives $\Gamma \vdash W$, and the per-task protocol membership gives $\text{Valid}(W)$. Outside the fragment—non-rejoining or multiway branches, type-misaligned rejoins, or cycles—no finite derivation in this three-constructor calculus need exist even when $(*)$ holds; there the edge-condition statement is the operative one, and it is exactly what the tool decides.

Both readings are *relative to the declared annotations*, and here that scoping is exact: contracts and typestate labels are never inferred—only the idempotence/persistence/effect annotations of Section 4 are—so on this tier Γ is the declared data and the three checks are unconditionally error-level. A workflow lacking a declaration is outside the theorem’s scope (the corresponding check is vacuous on that edge), not silently downgraded to a warning. $\square$

4 Downstream-aware Saga compensation

4.1 Model of failure and the post-commit region

Let W declare which tasks are *persistent* and, for each persistent P , a declared *compensator* C (the Saga pattern [3]). Every *fallible* state—a TASK, MAP, or PARALLEL—may raise the generic error `States.TaskFailed`; an error caught by a *broad* CATCH (one whose `ErrorEquals` contains `States.ALL` or `States.TaskFailed`) is routed to the catch target, otherwise it propagates to workflow failure. PARALLEL and MAP are fallible black boxes; a nested persistent task is analysed within its own sub-machine by the same procedure.

The completeness half of the guarantee below is stated relative to an explicit model of execution, made precise here so the theorem is neither over- nor under-claimed.

Assumption 2 (Failure model). *(A1) Single post-commit fault:* on each execution we reason about the *first* failure after P commits, and the compensation flow it triggers is itself fault-free—a standard single-fault Saga assumption. *(A2) Path feasibility:* CHOICE guards are abstracted, so every normal edge is feasible (the analysis is path-insensitive). *(A3) Unique generic catcher:* no fallible state has two CATCH clauses both matching `States.TaskFailed`, so the existential test over catchers coincides with ASL’s first-match resolution for the modeled error. *(A4) Linear handlers:* from a broad CATCH target, control follows a unique normal successor until it reaches C (as the typed DSL builds compensation chains), so “reaches C ” means “executes C ”.

Definition 5 (Post-commit region). $R(P)$ is the set of states reachable from P ’s normal successors by following *normal* (non-error) control-flow edges *without passing through* C . These are exactly the states that execute after P has committed its durable effect and before any compensation.

Definition 6 (Saga-completeness). A persistent task P with compensator C is *Saga-complete* when every execution in which P commits its effect and a subsequent *post-commit failure*

occurs—a fallible state in $R(P)$ raising the generic error, or a **Fail** terminal reached on a normal path in $R(P)$ —also executes C .

4.2 The SC4011 algorithm

Listing 1: Downstream-aware Saga check (per persistent task).

```

SC4011(W, P, C):
    region = post_commit_region(W, P, C)          # the region R(P)
    foreach Q in region:
        if is_fallible(Q) and not failure_reaches(Q, C):
            fire SC4011 at Q                      # a post-commit failure at Q
        escapes C
        if is_fail_terminal(Q):
            fire SC4011 at Q                      # a normal path commits P then ends
        in failure

post_commit_region(W, P, C):
    successors,
    return forward_closure(normal_successors(P),   # not passing through C
                           edges = normal, without = {C})

failure_reaches(Q, C):
    # Q's own generic failure is caught broadly and the handler reaches C by normal
    flow
    return exists c in catches(Q):
        broad(c.error_equals) and reaches(c.target, C, edges = normal)

is_fallible(Q):      return Q.kind in {Task, Map, Parallel}
is_fail_terminal(Q): return Q.kind == Fail
broad(errs):         return "States.ALL" in errs or "States.TaskFailed" in errs
reaches(s, C, e):    return C in forward_closure({s}, edges = e)

```

Theorem 3 (Compensation completeness under the declared model). *Fix a workflow W with declared persistence and compensators, and adopt Assumption 2. For a persistent task P with compensator C , SC4011 (Listing 1) emits no finding iff P is Saga-complete—iff every fallible $Q \in R(P)$ has a broad **Catch** whose target reaches C , and no normal path from P reaches a **Fail** terminal in $R(P)$. Under the model, SC4011 is a sound and complete decision procedure for Saga-completeness.*

Proof. Soundness (every finding is real, relative to A2). Suppose SC4011 fires. In the first case it fires at a fallible $Q \in R(P)$ with $\neg \text{failure_reaches}(Q, C)$. By A2 the normal path $P \rightarrow \dots \rightarrow Q$ is feasible, so some execution runs P to commitment and reaches Q ; being fallible, Q raises **States.TaskFailed** (A1: the single post-commit fault). By A3 the catcher the test inspects is the one ASL would match, and it is absent, non-broad, or routes to a handler not reaching C ; by A4 “reaches C ” is exact, so C does not execute. This is a concrete post-commit failure that leaks P ’s effect. In the second case SC4011 fires at a **Fail** terminal $Q \in R(P)$: the feasible normal path $P \rightarrow \dots \rightarrow Q$ commits P and then fails with no compensation—a more direct witness.

Completeness (every defect is found). Suppose P is not Saga-complete: some execution commits P and then a post-commit failure occurs without C . By A1 consider that first failure. If it is a fallible state Q , then $Q \in R(P)$; by A3 its generic error resolves to the single catcher the test inspects, and since C did not run, that catcher does not route to C (A4 makes “reaches” exact), so $\neg \text{failure_reaches}(Q, C)$ and SC4011 fires at Q . If instead the execution reaches a **Fail** terminal on a normal path without C , that terminal lies in $R(P)$ and SC4011 fires there. Conversely, if every fallible $Q \in R(P)$ has its (unique, by A3) broad catcher reaching C and no

normal path bypasses C to a `Fail`, then by A4 every post-commit fault routes into a chain that runs C , so P is Saga-complete. Hence SC4011 fires exactly on the non-Saga-complete workflows.

The guarantee is *relative to the declared facts*: a compensator is never *inferred*, so under pure inference the premise is absent and SC4011 stays silent (in contrast to the name-based, warning-level SC4010). $\square$

Remark 3 (Where the assumptions bite). Relaxing Assumption 2 costs completeness, not soundness. Without A1, a fault *inside* the compensation chain (e.g. a fallible compensator) can leak an earlier effect, which SC4011 does not model. Without A3, a fallible state whose first catcher matches `States.TaskFailed` but routes away from C , shadowing a later broad catcher, is missed. Without A4, a broad catcher whose handler *may* reach C on one branch and a `Fail` on another passes the existential test yet can skip C ; deciding this exactly needs C to *postdominate* the handler, which the check approximates by reachability. Non-generic errors (`States.Timeout`, service faults) caught by narrow catchers, and cross-branch aborts of a `Parallel/Map` that strand a sibling’s committed effect, are likewise outside the model. Closing these is future work; the corpus in the main paper satisfies A1–A4.

4.3 Reverse-order chains

Saga-completeness of a single (P, C) pair is local, but a multi-step Saga must unwind *all* prior effects on a late failure. Take persistent tasks $P_1, \dots, P_n$ committed in order with compensators $C_1, \dots, C_n$ wired as a *reverse-order chain*: each C_i ’s *unique* normal successor is C_{i-1} , and C_1 ’s is the failure terminal. Under this linear-chain hypothesis—exactly A4 of Assumption 2 applied to the chain—running C_i runs the whole prefix $C_i, C_{i-1}, \dots, C_1$ in order, so applying the single-pair check of Theorem 3 to each (P_i, C_i) certifies that any post-commit failure unwinds the full committed prefix. The hypothesis is essential: without the unique-successor wiring, per-pair reachability certifies neither the *order* nor the *completeness* of the unwind—a branch in the chain could skip some C_j while still reaching a later C_k . This is how the typed DSL builds a Saga by construction, which is why a correctly built DSL Saga raises no SC4011 and a one-edge break (redirecting one compensator to the failure terminal) makes SC4011 fire on precisely the upstream steps whose effects would then leak (the chain-break experiment in the main paper’s evaluation).

5 Conservative analyses: concurrency and temporal (notes)

The concurrency and temporal analyses are native but deliberately *conservative*: they under-approximate to avoid false alarms and make no soundness claim. All but one of their findings are warnings; the sole exception is the definite mis-configuration SC6003 (below), reported as an error.

Concurrency (SC5001/SC5002). `Parallel` branches and concurrent `Map` iterations each receive their own copy of the document, so the document cannot race; only *external* resources can. The check mines each task’s resource key—service plus the named target: `TableName`, `QueueUrl`, `TopicArn`, `Bucket`, `EventBusName`, ECS cluster/task identifiers, `Bedrock ModelId`, and nested-workflow `StateMachineArn/Name` pairs (statically named `states:startExecution` calls that provably share a child workflow and execution name also collide)—and considers only *overwriting* writes—`PutItem/PutObject`, which are last-writer-wins. For child workflows, the CloudFormation/SAM front end links sibling state machines through logical ids, `Ref/GetAtt`, `DefinitionSubstitutions`, and `StateMachineName`-derived ARNs, so the child’s own write set is recursively flattened when the definition is present in the artifact; externally deployed children with no local definition remain a scope boundary. A task counts as a write only when it is annotated non-idempotent (or persistent): an unannotated `PutItem` is not flagged, a deliberate

precision trade that keeps the analysis silent absent an effect declaration and contributes to its zero-false-positive record. The check also excludes `UpdateItem` (an atomic merge that commutes for disjoint attributes) and fire-and-forget `Publish/SendMessage` (independent messages). SC5001 flags two `Parallel` branches that overwrite the same resource; SC5002 a concurrent `Map` (`MaxConcurrency` $\neq 1$) whose iterations overwrite a shared resource while threading *no* per-item data (no `.$` reference in the write, so every iteration clobbers the same item). Restricting to overwriting writes with a provably shared key is what makes the analysis report a conflict only when two branches *provably* touch the same key—hence zero in-the-wild false positives, at the cost of not claiming completeness.

Temporal (SC6001–SC6003). SC6001 warns when a callback (`waitForTaskToken`) or Activity task has neither `TimeoutSeconds` nor `HeartbeatSeconds`, so it can wait up to the platform’s one-year maximum. SC6002 warns when a task’s worst-case retry budget—the geometric back-off sum $\sum_{k=0}^{a-1} interval \cdot backoff^k$ over a attempts (defaults $interval = 1s$, $backoff = 2$, $a = 3$)—exceeds a declared machine `TimeoutSeconds`. SC6003 is a definite mis-configuration: `HeartbeatSeconds` $\geq$ `TimeoutSeconds`, which the platform itself rejects only at deploy time. SC6001 is a warning because a missing timeout is occasionally acceptable for a short-lived task; SC6003 is the one error-level temporal finding, as it is unambiguous.

Remark 4 (Residual opacity beyond the modeled fragments). STEP CHECK tracks workflow variables and a JSONata fragment—direct `$states/$name` references and object literals—as first-class provenance, on equal footing with JSONPath: the document and variable shapes share one lattice and one fixpoint (§2.4). What remains lifted to $\top$ —soundly, at a cost in precision—is the genuinely opaque residue: JSONata functions, filters, and arithmetic (`$exists`, path predicates, `$merge`, string and math operators), the `$states.context` object, and the opaque results of external tasks. A richer JSONata value abstraction (for instance, tracking the keys `$merge` preserves) would recover some of this precision under the same lattice and fixpoint; it is the natural next step, and the shape machinery already accommodates it.

6 Artifact and implementation notes

This section records the implementation and reproduction detail deferred from the main paper’s implementation and artifact sections. The artifact contains the verifier, the corpora used in the paper, the mutation engine, the concrete execution oracle, the validator-panel scripts, the timing harness, the real-bug replay harness, and the scripts that regenerate the main paper’s figures and tables. The AWS corpus has 193 Step Functions workflows; the cross-format corpus has 66 CNCF Serverless Workflow examples.

Excluding tests, STEP CHECK is 4,552 lines of Rust, divided as follows: the workflow IR, 323 lines; the three frontends (ASL parse and emit, the typed DSL, and CNCF [7]), 784; annotation resolution and inference, 275; the eight analysis passes and their manager, 1,730; diagnostics, 115; the execution oracle, 269; the mutation engine, 316; and the command-line interface together with the statistics, evaluation, and oracle harnesses, 740; a further 383 lines of in-tree tests are not counted above. The data-flow pass is by far the largest analysis at 584 lines—the shape lattice of Section 2, its transfer functions, and the control-flow fixpoint—while the other seven passes range from roughly 85 to 240 lines each. (These counts are for the artifact revision and drift with continued development.)

Packaging and distribution. The artifact ships a continuous-integration pipeline that builds and tests the binary on Linux, macOS, and Windows on every change, and a release pipeline that packages it for language package managers—a `crates.io` crate (`cargo install stepcheck`) and an `npm` wrapper package that fetches the platform binary—so a Rust- or Node-centric CI

adopts it with one command; a copy-pasteable GitHub Actions gate is included. With `-json`, findings emit one stable object per finding carrying a code, severity, offending state, message, and remediation note (e.g. SC1101 at ChargeCard: “reads `$.total`, a field no execution reaching it can have produced,” noting that no state on any path binds it).

The command-line tool exposes `check`, `infer`, `scan`, `stats`, `eval`, `oracle`, `eval-pairs`, `mutate`, `emit`, and `demo`. `check` runs the eight passes; `emit` round-trips a workflow through the ASL emitter unchanged, the control form against which the multi-validator baseline of the main paper scores each injected defect; `oracle` runs the data-flow soundness cross-check; `eval-pairs` drives the real-bug benchmark by replaying each pre-fix and post-fix workflow; `mutate` injects one defect of a chosen class; and `demo` emits a built-in typed-DSL example to ASL.

The data-flow commands also expose the `-result-shapes` ablation used in the main paper’s evaluation. This flag leaves the default opaque-task transfer unchanged but, for the ablation run, treats declared task-output schemas and a small table of AWS service-result envelopes as closed records. On the strict-input missing-field mutation study, this improves SC1101 recall from 84/126 to 88/126; the paired bounded-loop oracle run confirms all 92 SC1101 reports emitted under the ablation with zero counterexamples.

The evaluation is reproducible offline on a commodity laptop. A Rust 1.96 toolchain builds the verifier and runs the mutation study, data-flow oracle, timing experiments, and real-bug replay. Node.js and Ruby drive the two local third-party validators used in the baseline (`asl-validator` and `statelint`), and Python renders the figures. No AWS account or credentials are required to reproduce the paper’s tables, figures, mutation results, data-flow oracle, timing results, or local validator baselines. AWS’s server-side `ValidateStateMachineDefinition` check is opt-in (`STEPCHECK_AWS=1`), creates no resources, and has recorded outputs committed with the artifact so its numbers reproduce offline. The only steps that touch an AWS account are the optional deployment checks reported in the main paper (Express, Standard, and live-callback runs; scripts `infra/deploy_run{,_standard,_callback}.sh`); no other result depends on them, and the captured execution evidence is committed with the artifact.

The ASL emitter is currently lossy for a workflow’s pure I/O-processing fields: among others it does not re-emit `InputPath`, `OutputPath`, or `ResultSelector`, nor the machine-level `TimeoutSeconds`; it also drops the `QueryLanguage` directive (a JSONata machine re-emits without it) and canonicalises a `Map`’s `ItemProcessor` to the older `Iterator` form. The AWS round-trip is therefore exercised on DSL-authored workflows that use none of these fields, where the deployed state machine is behaviourally equivalent to the verified IR.

7 Extended limitations

This section expands the threats-to-validity discussion of the main paper’s evaluation.

Corpus composition. The corpus comes from public AWS and CNCF sample repositories and skews small (median 6 states, maximum 33). We do not include proprietary industrial definitions because workflow files are sensitive artifacts: they expose resource names, account topology, and business-process structure, and no anonymization we are aware of preserves the semantic features the analyses consume (JSONPath references, resource keys, task names for inference). The generated chain, fan-out, and nested-`Map` sweeps exercise scale and topology, but they cannot stand in for industrial semantic diversity—richer contracts, deeper Saga chains, and organization-specific naming conventions that inference has not been tuned on.

Deployment check. The AWS deployment check exercises three execution models on StepCheck-verified definitions, each deployed unmodified (only `FunctionName` bound to a stand-in echo Lambda) and each reaching `SUCCEEDED` before teardown: (i) the DSL-authored order workflow as a synchronous *Express* machine; (ii) the same definition as a *Standard* machine

executed asynchronously, whose durable history retains all 24 events (Express retains none); and (iii) a DSL-verified order variant with a bounded `.waitForTaskToken` approval gate deployed as Standard (Express cannot run this integration), where the execution pauses at the callback, an external `SendTaskSuccess` resumes it, and it completes in 30 events. The callback carries a `TimeoutSeconds` bound, so the temporal pass raises no SC6001; removing the bound makes StepCheck flag it before deployment, tying the check to the deployed behaviour. To confirm the verified artifact runs correctly beyond a single execution, we additionally run the order workflow **100** times in each non-callback mode: all complete successfully (Express 100/100, Standard 100/100), with per-execution wall time of p50/p95/p99 $\approx 72/130/165$ ms (Express, mean ≈ 84 ms) and $\approx 345/456/484$ ms (Standard, mean ≈ 361 ms, five state transitions per run). We report these distributions as evidence of stable completion across execution models, not as a cross-mode AWS performance benchmark. StepCheck itself is also wired as a live gate: `.github/workflows/gate.yml` runs the blocking check (`--deny-warnings`) on every push/PR over the deployment artifacts and includes a negative test that injects a defect and fails the job unless the gate blocks it. What the deployment check still does *not* exercise is Activity workers, service-integration failures, or a broad multi-workflow performance benchmark; those behaviours remain covered by the static analyses and the offline oracle rather than by live execution.

Semantic drift. Declared premises are checked for syntactic rot (SC0011/SC0012) but not for truth: a sidecar that says a Lambda is idempotent when its implementation is not will pass the declared-tier checks. Generating the sidecar from code or infrastructure-as-code—the generated-metadata premise source—is the deployment pattern that keeps declared facts in sync; inference is the fallback and always stays at warning level.

References

- [1] Anonymous Author(s). Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics. Under submission to ICSOC, 2026. (The main paper this report accompanies.)
- [2] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.
- [3] H. Garcia-Molina and K. Salem. Sagas. In *ACM SIGMOD*, 1987.
- [4] R. E. Strom and S. Yemini. Typestate: a programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, 12(1), 1986.
- [5] R. DeLine and M. Fähndrich. Typestates for objects. In *ECOOP*, 2004.
- [6] S. Gössner, G. Normington, and C. Bormann (eds.). *RFC 9535: JSONPath — Query Expressions for JSON*. IETF, 2024.
- [7] Cloud Native Computing Foundation. *Serverless Workflow Specification*. <https://serverlessworkflow.io/>.
- [8] Amazon Web Services. *AWS Step Functions Developer Guide / Amazon States Language specification*. <https://states-language.net/spec.html>.